



ADITYA UNIVERSITY

SCHOOL OF ENGINEERING

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Cornerstone project work entitled “**Online Shopping System**” is submitted by

MALLELA SARIKA (25B11CS544)

V.BHAVYA SREE (25B11CSA19)

T.DEVI (25B11CS933)

PITHANI DIVYANJALI (26B21CS758)

in partial fulfillment of the requirements for the award of the B.Tech. degree in Computer Science and Engineering for the academic year 2026-2027.

Project Guide

Mrs.Dr. Vydehi
Assistant Professor,
Department of Computer Science

Head of the Department

Dr. A Phani Sridhar
Associate Professor,
Department of Computer Science

DECLARATION

We hereby declare that the Cornerstone project entitled “**Online Shopping System**” is original and is submitted to **ADITYA UNIVERSITY**, Surampalem, in partial fulfillment of the requirements for the award of the **B.Tech. Degree in Computer Science and Engineering**. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy

Place: Surampalem

By

M.Sarika	(25B11CS544)
V.Bhavya Sri	(25B11CSA19)
T.Devi	(25B11CS933)
P.Divyanjali	(26B21CS758)

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, Mrs.Dr. Vydehi , **Assistant Professor**, who has guided us throughout the project. His valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, **Dr. K Chandra Sekhar, Associate Professor and Dr. Hari Jyothula, Associate Professor and Dr. P Siva Satya Prasad**, Department of Computer Science and Engineering, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department **Dr. A Phani Sridhar, Associate Professor**, for his inspiration and for providing all the necessary facilities and resources required for this project.

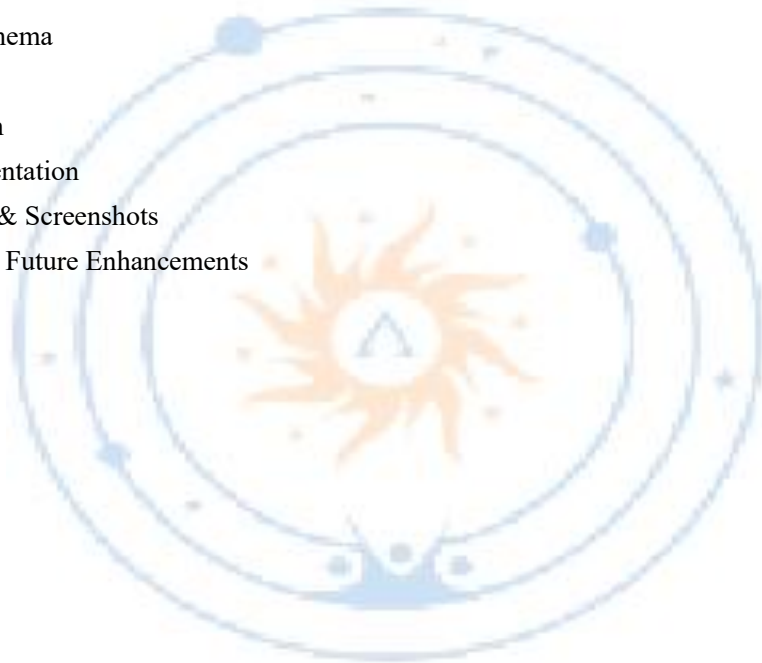
We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean, School of Computing** for their encouragement and support during the course of our project.

We would like to express our sincere gratitude to **Dr. G. Suresh, Registrar, Dr. A Ramesh, Pro Vice-Chancellor (Engineering & Sciences), Dr. S. Rama Sree, Pro Vice-Chancellor (Academics), Dr. M.B. Srinivas, Vice-Chancellor, Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor and Management, Aditya University** for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **faculty members, lab technicians, non-teaching staff, and our friends** who have directly or indirectly supported us in completing this Cornerstone project successfully.

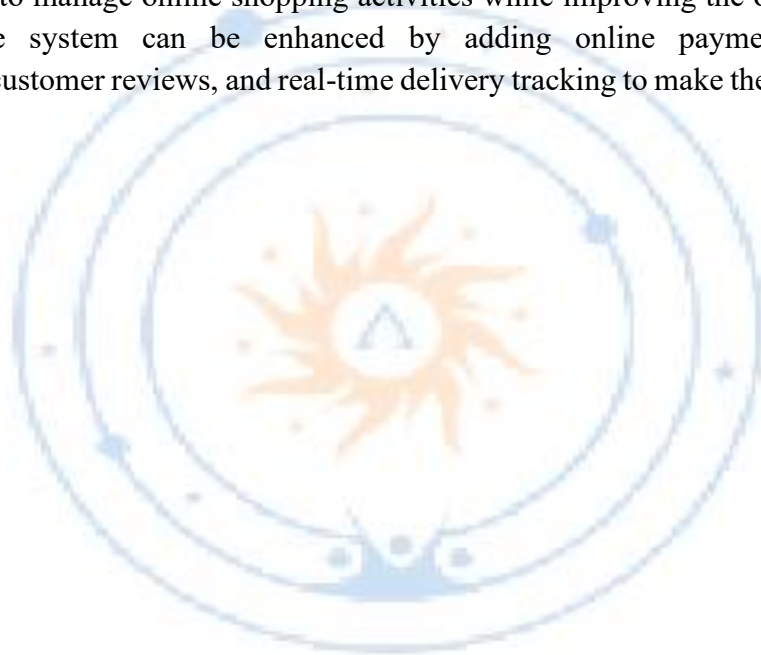
TABLE OF CONTENTS

1. Cover Page
2. Certificate
3. Acknowledgement
4. Table of Contents
5. Abstract
6. Introduction
7. System Analysis
8. Database Design
9. ER Diagram
10. Relational Schema
11. Table Design
12. Normalization
13. SQL Implementation
14. SQL Queries & Screenshots
15. Conclusion & Future Enhancements



ABSTRACT

+Nowadays, online shopping has become one of the most popular and convenient ways to purchase products. Customers prefer shopping online because it saves time, offers a wide range of products, and provides easy payment options. The main objective of this project is to develop an Online Shopping System that allows users to browse products, place orders, make payments, and track their purchases through a simple and user-friendly platform. The proposed system includes modules such as user registration, product management, shopping cart, order management, payment processing, and order tracking. Customers can search for products, add them to the cart, place orders, and view their order history. The database is designed using proper tables, primary keys, foreign keys, and relationships to ensure data accuracy and reduce redundancy through normalization. This project provides an efficient and organized way to manage online shopping activities while improving the overall user experience. In the future, the system can be enhanced by adding online payment gateways, product recommendations, customer reviews, and real-time delivery tracking to make the platform more reliable and user-friendly.



LIST OF FIGURES

Figure 1. ER Diagram of Online Shopping System

Figure 2. Proposed Three-Layer Database Application Architecture

Figure 3. Product Management Screen – Placeholder

Figure 4. Customer and Order Management Screen – Placeholder

Figure 5. Shopping Cart and Checkout Screen – Placeholder

Figure 6. Order Tracking and Report Generation Screen – Placeholder



ACRONYMS

- DBMS – Database Management System
- ER – Entity Relationship
- SQL – Structured Query Language
- DDL – Data Definition Language
- DQL – Data Query Language
- DCL – Data Control Language
- TCL – Transaction Control Language
- PK – Primary Key
- FK – Foreign Key
- 1NF – First Normal Form
- 2NF – Second Normal Form
- 3NF – Third Normal Form



1. INTRODUCTION

1.1 Brief Information about the Project

An Online Shopping System is a database application designed to manage online shopping and e-commerce processes. The project provides a centralized relational database for customers, products, categories, orders, order items, payments, and reviews. It is intended to replace scattered manual records and multiple files with a structured database that supports consistent data storage, efficient retrieval, and systematic online shopping processin

1.2 Motivation and Contribution of Project

The project is motivated by common problems in online shopping such as duplicate records, time-consuming product searches, inconsistent information, difficulty in tracking orders, and challenges in maintaining customer and inventory records. A centralized database provides a systematic approach to storing related shopping information and enforcing relationships through keys and constraints.

- Centralizing shopping-related records in one relational database.
- Reducing duplicate and inconsistent customer information.
- Maintaining referential integrity using primary and foreign keys.
- Supporting faster product searches and order tracking.
- Providing a foundation for customer, product, category, order, payment, and review management.

1.3 Objectives of the Project

- Develop a centralized online shopping database.
- Manage customer records.
- Store and manage product and category details.
- Maintain order and order item information.
- Handle order placement and customer processing.
- Record and manage payment details.

1.4 Scope of the Project

The scope covers database-oriented management of online shopping information. The core scope includes customer, product, category, order, order item, payment, and review data. The database can be used as the backend for an online shopping management system and can be extended later with product search interfaces, inventory updates, order tracking, customer reviews, and sales reporting.

2. SYSTEM ANALYSIS

2.1 Existing System

- Manual product and order record maintenance.
- Multiple files or disconnected records.
- Duplicate customer records and inconsistent information.
- Time-consuming product and order searches.
- Difficulty in tracking order status.
- Difficulty in maintaining inventory and payment records.
- Higher possibility of data-entry errors.

2.2 Proposed System

The proposed system uses a centralized relational database. It organizes online shopping information into related tables and uses primary keys, foreign keys, uniqueness rules, NOT NULL conditions, and CHECK constraints where appropriate. The system supports customer, product, category, order, order item, payment, and review management.

- Centralized database
- Customer management
- Product management
- Category management
- Order management
- Order item management
- Payment management
- SQL-based reporting
- Referential integrity

2.3 Functional Requirements

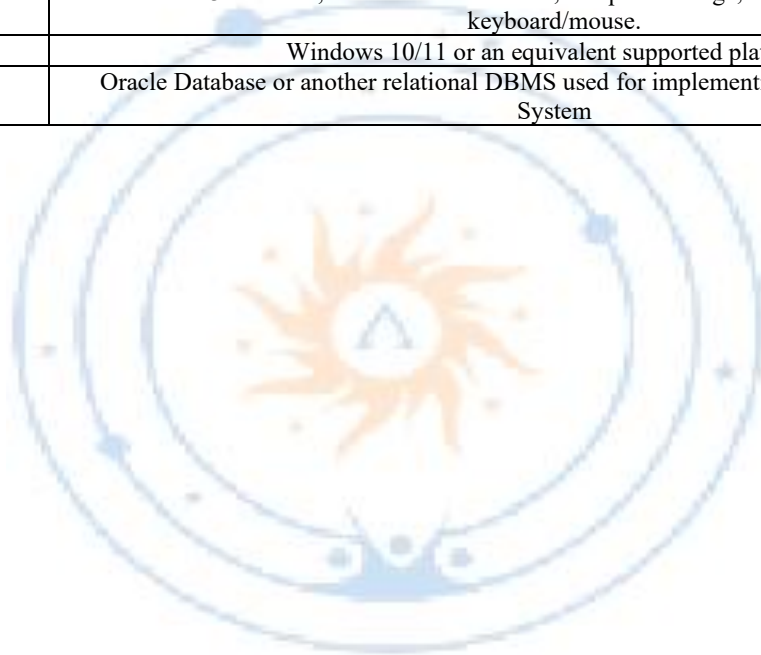
Module	Functional Requirement
Customer Management	Add, update, search, and display customer details.
Product Management	Store, update, search, and display product details.
Category Management	Store and manage product category details.
Order Management	Place, maintain and track customer order details.
Order Item Management	Store product, quantity, and price details for each order.
Payment Management	Record payment amount, payment method, and payment status.
Review Management	Store customer ratings and review details for products.
Search and Reporting	Retrieve customer, product, order, payment, and review information using SQL queries.

2.4 Non-Functional Requirements

- · Performance – queries should return results efficiently for normal online shopping workloads.
- · Integrity – invalid references and duplicate key values should be prevented.
- · Security – database privileges and user access should be controlled for authorized users.
- · Usability – product information and reports should be understandable to users.
- · Reliability – transactions should preserve consistent database state.
- · Scalability – the design should support growth in customers, products, orders, and transactions.

2.5 Requirements Specification

Category	Suggested Requirement
Hardware	Intel i3 or above; 4 GB RAM minimum; adequate storage; standard display and keyboard/mouse.
Operating System	Windows 10/11 or an equivalent supported platform.
Database	Oracle Database or another relational DBMS used for implementing the Online Shopping System



3. DATABASE DESIGN

3.1 Major Entities

Entity	Description
Customer	Stores customer personal and contact information.
Product	Stores product details, description, price, stock, and category information.
Order	Stores order details, order date, total amount, and order status.
OrderItem	Stores details of products included in an order, including quantity and price.
Category	Stores product category information and description.
Payment	Stores applicant and document verification details.
Review	Stores issued passport details and status.

3.2 ER Model

The ER model represents the major entities and their relationships in the Online Shopping System. The main entities include Customer, Product, Order, OrderItem, Category, Payment, and Review. The model represents relationships such as Customer–Order, Customer–Review, Order–OrderItem, Product–OrderItem, Product–Category, Product–Review, and Order–Payment. The diagram also includes attributes such as Customer_ID, Product_ID, Order_ID, OrderItem_ID, Category_ID, Payment_ID, Review_ID, and other related online shopping attributes.

3.3 Relationships

Relationship	Description
Customer → Order	One customer can place one or more orders.
Customer → Review	One customer can write one or more reviews.
Order → OrderItem	One order can contain one or more order items.
Product → Category	One product can be listed in one or more order items.
Product → Review	One product can receive one or more reviews.
Order → Payment	Each order has an associated payment record.

4. ER DIAGRAM

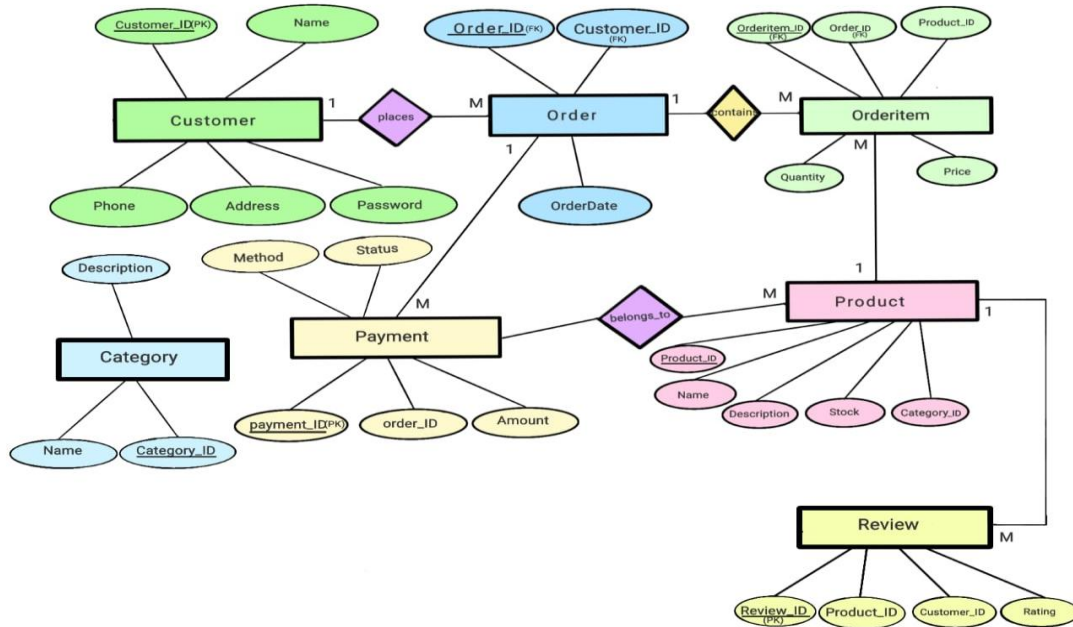


Figure 1: ER Diagram of Passport Application Management System

The ER diagram summarizes the major entities, attributes, relationships, and cardinalities of the Online Shopping System. The diagram represents entities such as Customer, Order, OrderItem, Product, Category, Payment, and Review, along with their relationships. It provides a structured representation of the database design and shows how online shopping information is connected across the different entities.

5. RELATIONAL SCHEMA

Relation	Relational Schema
CUSTOMER	Customer_ID (PK), Name, Email, Phone, Address, Password
PRODUCT	Product_ID (PK), Name, Description, Price, Stock, Category_ID (FK)
ORDERS	Order_ID (PK), Customer_ID (FK), OrderDate, Total, Status
ORDERITEM	OrderItem_ID (PK), Order_ID (FK), Product_ID (FK), Quantity, Price
CATEGORY	Category_ID (PK), Name, Description
PAYMENT	Payment_ID (PK), Order_ID (FK), Amount, Method, Status
REVIEW	Review_ID (PK), Product_ID (FK), Customer_ID (FK), Rating, Comment

The relational schema represents the structure of the Online Shopping System. The Order relation references Customer; Product references Category; OrderItem references Order and Product; Payment references Order; and Review references Product and Customer. Primary keys uniquely identify records in each relation, while foreign keys establish relationships between the related tables and maintain referential integrity.



6. TABLE DESIGN

6.1 Customer Table

Attribute	Data Type	Constraint	Description
Customer ID	INT / NUMBER	PK, NOT NULL	Unique applicant identifier
Name	VARCHAR	NOT NULL	Customer full name
Email	VARCHAR	UNIQUE	Applicant date of birth
Phone	VARCHAR	UNIQUE	Customer contact number
Address	VARCHAR	-	Customer residential address
Password	VARCHAR	UNOT NULL	Customer login password

6.2 Order Table

Attribute	Data Type	Constraint	Description
Order ID	INT	PK, NOT NULL	Unique order identifier
Customer ID	INT	FK, NOT NULL	References Customer
OrderDate	DATE	NOT NULL	Date of order
Total	NUMBER	-	Total order amount
Status	VARCHAR	-	Current order status

6.3 OrderItem Table

Attribute	Data Type	Constraint	Description
Orderitem ID	INT	PK, NOT NULL	Unique order item identifier
Order ID	INT	FK, NOT NULL	References Order
Product ID	INT	FK,NOT NULL	Reference Product
Quantity	INT	NOT NULL	Quantity of the product ordered
Price	NUMBER	NOT NULL	Price of the product in the order

6.4 Keys and Constraints

Constraint Type	Applied To / Purpose
Primary Key	Customer_ID, Product_ID, Order_ID, OrderItem_ID, Category_ID, Payment_ID, Review_ID
Foreign Key	Product.Category_ID, Order.Customer_ID, OrderItem.Order_ID, OrderItem.Product_ID, Payment.Order_ID, Review.Product_ID, Review.Customer_ID
NOT NULL	Required attributes such as names, identifiers, application dates, and required references
UNIQUE	Customer email/phone and other attributes where unique identification is required
CHECK	Values such as product stock, payment amount, rating, and payment status can be restricted to valid values
DEFAULT	Can be used for fields such as payment status, order status, or status fields where appropriate

7. NORMALIZATION

Normalization organizes data into related relations to reduce redundancy and update anomalies. The Online Shopping System is designed to organize customer, product, order, cart, payment, delivery, review, and category information into separate related tables. The following example illustrates the normalization process from UNF to 3NF.

7.1 Unnormalized Form (UNF)

A 7.1 Unnormalized Form (UNF)

A single online shopping record might contain Customer_ID, Customer_Name, Email, Order_ID, Product_ID, Product_Name, Category, Quantity, Price, Payment_Amount, Delivery_Status, and Review_Rating in one record. Multiple product details and repeated order-related information can create redundancy.

7.2 First Normal Form (1NF)

In 1NF, repeating groups are removed and each attribute contains atomic values. Each product, payment, delivery, and review record is represented separately instead of storing multiple values in a single field. Customer and order details may still be repeated across multiple related records.

7.3 Second Normal Form (2NF)

In 2NF, partial dependencies on a composite key are removed. Customer-specific attributes depend on Customer_ID, product-specific attributes depend on Product_ID, order-specific attributes depend on Order_ID, and other details depend on their respective primary keys. These dependencies motivate the separation of Customer, Product, Order, Cart, Payment, Delivery, Review, and Category relations.

7.4 Third Normal Form (3NF)

In 3NF, transitive dependencies are removed. Customer details are stored in the Customer relation rather than being repeated in Order. Product details are stored in the Product relation, while cart, payment, delivery, review, and category details are stored in their respective relations. Foreign keys are used to establish relationships between these tables. The resulting structure reduces redundancy and supports data integrity.

Relation	Reason for Separation
Customer	Stores customer-specific personal and contact attributes once.
Product	Stores product-specific information such as name, price, and stock
Order	Stores order-specific details and references the customer
Cart	Stores shopping cart details associated with a customer and products.
Payment	Stores payment information associated with an order.
Delivery	Stores delivery-specific details associated with an order.
Review	Stores product review and rating information associated with a customer and product.

8. SQL IMPLEMENTATION

8.1 DDL – Data Definition Language

The following DDL statements define the structure of the Passport Application Management System database. The tables are created using primary keys, foreign keys, NOT NULL, UNIQUE, CHECK, and DEFAULT constraints where appropriate. The implementation provides a structured relational database for managing applicants, passport applications, documents, appointments, payments, verification, and passport records.

```
DROP TABLE REVIEW;
DROP TABLE PAYMENT;
DROP TABLE ORDERITEM;
DROP TABLE ORDERS;
DROP TABLE PRODUCT;
DROP TABLE CUSTOMER;
DROP TABLE CATEGORY;

CREATE TABLE CATEGORY (
  Category_ID NUMBER,
  Name VARCHAR2(50),
  Description VARCHAR2(100),
  PRIMARY KEY (Category_ID)
);

CREATE TABLE CUSTOMER (
  Customer_ID NUMBER,
  Name VARCHAR2(50),
  Phone VARCHAR2(15),
  Address VARCHAR2(100),
  Password VARCHAR2(50),
  PRIMARY KEY (Customer_ID)
);

CREATE TABLE PRODUCT (
  Product_ID NUMBER,
  Name VARCHAR2(50),
  Description VARCHAR2(100),
  Stock NUMBER,
  Category_ID NUMBER,
  PRIMARY KEY (Product_ID),
  FOREIGN KEY (Category_ID) REFERENCES CATEGORY(Category_ID)
);

CREATE TABLE ORDERS (
  Order_ID NUMBER,
  Customer_ID NUMBER,
  OrderDate DATE,
  PRIMARY KEY (Order_ID),
  FOREIGN KEY (Customer_ID) REFERENCES CUSTOMER(Customer_ID)
);

CREATE TABLE ORDERITEM (
  OrderItem_ID NUMBER,
```

```

Order_ID NUMBER,
Product_ID NUMBER,
Quantity NUMBER,
Price NUMBER,
PRIMARY KEY (OrderItem_ID),
FOREIGN KEY (Order_ID) REFERENCES ORDERS(Order_ID),
FOREIGN KEY (Product_ID) REFERENCES PRODUCT(Product_ID)
);

```

```

CREATE TABLE PAYMENT (
Payment_ID NUMBER,
Order_ID NUMBER,
Amount NUMBER,
Method VARCHAR2(20),
Status VARCHAR2(20),
PRIMARY KEY (Payment_ID),
FOREIGN KEY (Order_ID) REFERENCES ORDERS(Order_ID)
);

```

```

CREATE TABLE REVIEW (
Review_ID NUMBER,
Product_ID NUMBER,
Customer_ID NUMBER,
Rating NUMBER,
PRIMARY KEY (Review_ID),
FOREIGN KEY (Product_ID) REFERENCES PRODUCT(Product_ID),
FOREIGN KEY (Customer_ID) REFERENCES CUSTOMER(Customer_ID)
);

```

```

INSERT INTO CATEGORY VALUES (1,'Electronics','Electronic devices');
INSERT INTO CATEGORY VALUES (2,'Mobiles','Mobile phones');
INSERT INTO CATEGORY VALUES (3,'Laptops','Laptop computers');
INSERT INTO CATEGORY VALUES (4,'Clothing','Men and women clothing');
INSERT INTO CATEGORY VALUES (5,'Shoes','Footwear products');
INSERT INTO CATEGORY VALUES (6,'Books','Educational books');
INSERT INTO CATEGORY VALUES (7,'Beauty','Beauty products');
INSERT INTO CATEGORY VALUES (8,'Sports','Sports equipment');
INSERT INTO CATEGORY VALUES (9,'Furniture','Home furniture');
INSERT INTO CATEGORY VALUES (10,'Grocery','Daily grocery items');
INSERT INTO CATEGORY VALUES (11,'Watches','Wrist watches');
INSERT INTO CATEGORY VALUES (12,'Bags','Bags and backpacks');
INSERT INTO CATEGORY VALUES (13,'Toys','Kids toys');
INSERT INTO CATEGORY VALUES (14,'Kitchen','Kitchen products');
INSERT INTO CATEGORY VALUES (15,'Accessories','Electronic accessories');

```

```

SELECT * FROM CATEGORY;

```

```

INSERT INTO CUSTOMER VALUES (1,'Ananya','9876543210','Hyderabad','Pass101');
INSERT INTO CUSTOMER VALUES (2,'Rahul','9876543211','Vijayawada','Pass102');
INSERT INTO CUSTOMER VALUES (3,'Sneha','9876543212','Chennai','Pass103');
INSERT INTO CUSTOMER VALUES (4,'Arjun','9876543213','Bangalore','Pass104');
INSERT INTO CUSTOMER VALUES (5,'Priya','9876543214','Vizag','Pass105');
INSERT INTO CUSTOMER VALUES (6,'Kiran','9876543215','Guntur','Pass106');
INSERT INTO CUSTOMER VALUES (7,'Divya','9876543216','Rajahmundry','Pass107');
INSERT INTO CUSTOMER VALUES (8,'Ravi','9876543217','Tirupati','Pass108');
INSERT INTO CUSTOMER VALUES (9,'Keerthi','9876543218','Warangal','Pass109');

```

```

INSERT INTO CUSTOMER VALUES (10,'Varun','9876543219','Nellore','Pass110');
INSERT INTO CUSTOMER VALUES (11,'Pooja','9876543220','Kakinada','Pass111');
INSERT INTO CUSTOMER VALUES (12,'Suresh','9876543221','Kurnool','Pass112');
INSERT INTO CUSTOMER VALUES (13,'Meena','9876543222','Ongole','Pass113');
INSERT INTO CUSTOMER VALUES (14,'Vikas','9876543223','Machilipatnam','Pass114');
INSERT INTO CUSTOMER VALUES (15,'Harini','9876543224','Eluru','Pass115');

SELECT * FROM CUSTOMER;

INSERT INTO PRODUCT VALUES (1,'Smart TV','43 inch LED TV',20,1);
INSERT INTO PRODUCT VALUES (2,'iPhone 15','Apple smartphone',15,2);
INSERT INTO PRODUCT VALUES (3,'Dell Laptop','Core i5 laptop',10,3);
INSERT INTO PRODUCT VALUES (4,'T Shirt','Cotton T Shirt',30,4);
INSERT INTO PRODUCT VALUES (5,'Running Shoes','Sports running shoes',25,5);
INSERT INTO PRODUCT VALUES (6,'DBMS Book','Database textbook',40,6);
INSERT INTO PRODUCT VALUES (7,'Face Cream','Skin care cream',35,7);
INSERT INTO PRODUCT VALUES (8,'Cricket Bat','Professional cricket bat',12,8);
INSERT INTO PRODUCT VALUES (9,'Office Chair','Comfortable office chair',18,9);
INSERT INTO PRODUCT VALUES (10,'Rice Bag','25 kg rice bag',50,10);
INSERT INTO PRODUCT VALUES (11,'Wrist Watch','Digital wrist watch',22,11);
INSERT INTO PRODUCT VALUES (12,'Backpack','College backpack',28,12);
INSERT INTO PRODUCT VALUES (13,'Toy Car','Remote control toy car',16,13);
INSERT INTO PRODUCT VALUES (14,'Mixer Grinder','Kitchen mixer grinder',14,14);
INSERT INTO PRODUCT VALUES (15,'Wireless Mouse','Bluetooth wireless mouse',32,15);

SELECT * FROM PRODUCT;

INSERT INTO ORDERS VALUES (1,1,TO_DATE('2026-09-01','YYYY-MM-DD'));
INSERT INTO ORDERS VALUES (2,2,TO_DATE('2026-09-02','YYYY-MM-DD'));
INSERT INTO ORDERS VALUES (3,3,TO_DATE('2026-09-03','YYYY-MM-DD'));
INSERT INTO ORDERS VALUES (4,4,TO_DATE('2026-09-04','YYYY-MM-DD'));
INSERT INTO ORDERS VALUES (5,5,TO_DATE('2026-09-05','YYYY-MM-DD'));
INSERT INTO ORDERS VALUES (6,6,TO_DATE('2026-09-06','YYYY-MM-DD'));
INSERT INTO ORDERS VALUES (7,7,TO_DATE('2026-09-07','YYYY-MM-DD'));
INSERT INTO ORDERS VALUES (8,8,TO_DATE('2026-09-08','YYYY-MM-DD'));
INSERT INTO ORDERS VALUES (9,9,TO_DATE('2026-09-09','YYYY-MM-DD'));
INSERT INTO ORDERS VALUES (10,10,TO_DATE('2026-09-10','YYYY-MM-DD'));
INSERT INTO ORDERS VALUES (11,11,TO_DATE('2026-09-11','YYYY-MM-DD'));
INSERT INTO ORDERS VALUES (12,12,TO_DATE('2026-09-12','YYYY-MM-DD'));
INSERT INTO ORDERS VALUES (13,13,TO_DATE('2026-09-13','YYYY-MM-DD'));
INSERT INTO ORDERS VALUES (14,14,TO_DATE('2026-09-14','YYYY-MM-DD'));
INSERT INTO ORDERS VALUES (15,15,TO_DATE('2026-09-15','YYYY-MM-DD'));

SELECT * FROM ORDERS;

INSERT INTO ORDERITEM VALUES (1,1,1,1,35000);
INSERT INTO ORDERITEM VALUES (2,2,2,1,65000);
INSERT INTO ORDERITEM VALUES (3,3,3,1,55000);
INSERT INTO ORDERITEM VALUES (4,4,4,2,1200);
INSERT INTO ORDERITEM VALUES (5,5,5,1,2500);
INSERT INTO ORDERITEM VALUES (6,6,6,1,800);
INSERT INTO ORDERITEM VALUES (7,7,7,2,900);
INSERT INTO ORDERITEM VALUES (8,8,8,1,3000);
INSERT INTO ORDERITEM VALUES (9,9,9,1,6000);
INSERT INTO ORDERITEM VALUES (10,10,10,2,1500);
INSERT INTO ORDERITEM VALUES (11,11,11,1,2000);

```

```
INSERT INTO ORDERITEM VALUES (12,12,12,1,1800);
INSERT INTO ORDERITEM VALUES (13,13,13,2,1600);
INSERT INTO ORDERITEM VALUES (14,14,14,1,4500);
INSERT INTO ORDERITEM VALUES (15,15,15,1,1200);
```

```
SELECT * FROM ORDERITEM;
```

```
INSERT INTO PAYMENT VALUES (1,1,35000,'UPI','Paid');
INSERT INTO PAYMENT VALUES (2,2,65000,'Card','Paid');
INSERT INTO PAYMENT VALUES (3,3,55000,'Net Banking','Paid');
INSERT INTO PAYMENT VALUES (4,4,1200,'UPI','Paid');
INSERT INTO PAYMENT VALUES (5,5,2500,'Card','Paid');
INSERT INTO PAYMENT VALUES (6,6,800,'Cash','Pending');
INSERT INTO PAYMENT VALUES (7,7,1800,'UPI','Paid');
INSERT INTO PAYMENT VALUES (8,8,3000,'Card','Paid');
INSERT INTO PAYMENT VALUES (9,9,6000,'Net Banking','Paid');
INSERT INTO PAYMENT VALUES (10,10,3000,'UPI','Pending');
INSERT INTO PAYMENT VALUES (11,11,2000,'Card','Paid');
INSERT INTO PAYMENT VALUES (12,12,1800,'UPI','Paid');
INSERT INTO PAYMENT VALUES (13,13,3200,'Cash','Pending');
INSERT INTO PAYMENT VALUES (14,14,4500,'Card','Paid');
INSERT INTO PAYMENT VALUES (15,15,1200,'UPI','Paid');
```

```
SELECT * FROM PAYMENT;
```

```
INSERT INTO REVIEW VALUES (1,1,1,5);
INSERT INTO REVIEW VALUES (2,2,2,4);
INSERT INTO REVIEW VALUES (3,3,3,5);
INSERT INTO REVIEW VALUES (4,4,4,4);
INSERT INTO REVIEW VALUES (5,5,5,5);
INSERT INTO REVIEW VALUES (6,6,6,4);
INSERT INTO REVIEW VALUES (7,7,7,3);
INSERT INTO REVIEW VALUES (8,8,8,5);
INSERT INTO REVIEW VALUES (9,9,9,4);
INSERT INTO REVIEW VALUES (10,10,10,3);
INSERT INTO REVIEW VALUES (11,11,11,5);
INSERT INTO REVIEW VALUES (12,12,12,4);
INSERT INTO REVIEW VALUES (13,13,13,5);
INSERT INTO REVIEW VALUES (14,14,14,4);
INSERT INTO REVIEW VALUES (15,15,15,5);
```

```
SELECT * FROM REVIEW;
```

```
SELECT COUNT(*) FROM CATEGORY;
SELECT COUNT(*) FROM CUSTOMER;
SELECT COUNT(*) FROM PRODUCT;
SELECT COUNT(*) FROM ORDERS;
SELECT COUNT(*) FROM ORDERITEM;
SELECT COUNT(*) FROM PAYMENT;
SELECT COUNT(*) FROM REVIEW;
```

8.2 DQL – Data Query Language

```
SELECT * FROM CUSTOMER;
```

```
SELECT Customer_ID, Name, Email
FROM CUSTOMER
WHERE Address = 'Hyderabad';
SELECT Order_ID, Customer_ID, OrderDate, Status
FROM ORDERS
ORDER BY OrderDate;
SELECT C.Name, O.Order_ID, O.Status
FROM CUSTOMER C
JOIN ORDERS O
ON C.Customer_ID = O.Customer_ID;
SELECT C.Name, O.Order_ID, P.Name AS Product_Name,
      OI.Quantity, OI.Price
FROM CUSTOMER C
JOIN ORDERS O
ON C.Customer_ID = O.Customer_ID
JOIN ORDERITEM OI
ON O.Order_ID = OI.Order_ID
JOIN PRODUCT P
ON OI.Product_ID = P.Product_ID;
```

Sample INSERT Statements

```
INSERT INTO PRODUCT
(Product_ID, Name, Description, Stock, Category_ID)
VALUES
(2, 'Laptop', '15 inch Laptop', 10, 1);
```

```
INSERT INTO PRODUCT
(Product_ID, Name, Description, Stock, Category_ID)
VALUES
(3, 'Headphones', 'Wireless Headphones', 25, 1);
```

```
INSERT INTO PRODUCT
(Product_ID, Name, Description, Stock, Category_ID)
VALUES
(4, 'Smartphone', 'Android Smartphone', 15, 1);
```

```
INSERT INTO PRODUCT
(Product_ID, Name, Description, Stock, Category_ID)
VALUES
(5, 'Keyboard', 'Wireless Keyboard', 30, 1);
```

```
INSERT INTO PRODUCT
(Product_ID, Name, Description, Stock, Category_ID)
VALUES
(6, 'Mouse', 'Wireless Mouse', 20, 1);
```

9. SQL QUERIES & SCREENSHOTS

The following query set is prepared to satisfy the documentation requirement for a 30–40 query SQL section. The examples cover selection, filtering, joins, aggregation, subqueries, updates, deletion, and reporting. Exact outputs depend on the records inserted in the final Online Shopping System database.

Q1. Display products whose stock is greater than 20.

```
SELECT Product_ID, Name, Stock FROM PRODUCT WHERE Stock > 20;
```

Output:



PRODUCT_ID	NAME	STOCK
4	T Shirt	30
5	Running Shoes	25
6	DBMS Book	40
7	Face Cream	35
10	Rice Bag	50
11	Wrist Watch	22
12	Backpack	28
15	Wireless Mouse	32

Q2. Display the payment details of orders whose payment is pending.

```
SELECT Payment_ID, Order_ID, Amount  
FROM PAYMENT WHERE Status = 'Pending';
```

Output:

PAYMENT_ID	ORDER_ID	AMOUNT
6	6	800
10	10	3000
13	13	3200

Q3. Display customers whose address is Hyderabad.

```
SELECT Customer_ID, Name, Phone FROM  
CUSTOMER WHERE Address = 'Hyderabad';
```

Output:

CUSTOMER_ID	NAME	PHONE
1	Ananya	9876543210

Q4. Display products according to their stock in ascending order.

```
SELECT Product_ID, Name, Stock FROM PRODUCT ORDER BY Stock ASC;
```

Output:

PRODUCT_ID	NAME	STOCK
3	Dell Laptop	10
8	Cricket Bat	12
14	Mixer Grinder	14
2	iPhone 15	15
13	Toy Car	16
9	Office Chair	18
1	Smart TV	20
11	Wrist Watch	22
5	Running Shoes	25
12	Backpack	28
4	T Shirt	30
15	Wireless Mouse	32
7	Face Cream	35
6	DBMS Book	40
10	Rice Bag	50

Q5. Display the customer name and the date on which they placed the order.

```
SELECT C.Name, O.OrderDate FROM CUSTOMER C JOIN ORDERS O ON C.Customer_ID = O.Customer_ID;
```

Output:

NAME	ORDERDATE
Ananya	01-SEP-26
Rahul	02-SEP-26
Sneha	03-SEP-26
Arjun	04-SEP-26
Priya	05-SEP-26
Kiran	06-SEP-26
Divya	07-SEP-26
Ravi	08-SEP-26
Keerthi	09-SEP-26
Varun	10-SEP-26
Pooja	11-SEP-26
Suresh	12-SEP-26
Meena	13-SEP-26
Vikas	14-SEP-26
Harini	15-SEP-26

Q6. Find the highest available stock among all products.

```
SELECT MAX(Stock) AS Highest_Stock  
  
FROM PRODUCT;
```

Output:

HIGHEST_STOCK
50

Q7. Display the product name along with its category name.

```
SELECT P.Name AS Product_Name,C.Name AS Category_Name FROM PRODUCT P  
  
JOIN CATEGORY C  
ON P.Category_ID = C.Category_ID;
```

Output:

PRODUCT_NAME	CATEGORY_NAME
Smart TV	Electronics
iPhone 15	Mobiles
Dell Laptop	Laptops
T Shirt	Clothing
Running Shoes	Shoes
DBMS Book	Books
Face Cream	Beauty
Cricket Bat	Sports
Office Chair	Furniture
Rice Bag	Grocery
Wrist Watch	Watches
Backpack	Bags
Toy Car	Toys
Mixer Grinder	Kitchen
Wireless Mouse	Accessories

Q8. Find the number of orders placed by each customer.

```

SELECT C.Name, COUNT(O.Order_ID) AS Total_Orders FROM CUSTOMER C

JOIN ORDERS O
ON C.Customer_ID = O.Customer_ID
GROUP BY C.Name;

```

Output:

NAME	TOTAL_ORDERS
Kiran	1
Meena	1
Arjun	1
Ananya	1
Sneha	1
Varun	1
Suresh	1
Rahul	1
Priya	1
Divya	1
Ravi	1
Vikas	1
Keerthi	1
Pooja	1
Harini	1

Q9. Find the average rating given to all products.

```

SELECT ROUND(AVG(Rating),2) AS Average_Rating

FROM REVIEW;

```

Output:

AVERAGE_RATING
4.33

Q10. Display each product along with the rating given to it.

```

SELECT P.Name AS Product_Name, R.Rating

FROM PRODUCT P
JOIN REVIEW R
ON P.Product_ID = R.Product_ID;

```

Output:

PRODUCT_NAME	RATING
Smart TV	5
iPhone 15	4
Dell Laptop	5
T Shirt	4
Running Shoes	5
DBMS Book	4
Face Cream	3
Cricket Bat	5
Office Chair	4
Rice Bag	3
Wrist Watch	5
Backpack	4
Toy Car	5
Mixer Grinder	4
Wireless Mouse	5

Q11. Find the total payment amount for each payment method.

```
SELECT Method, SUM(Amount) AS Total_Amount FROM PAYMENT  
GROUP BY Method;
```

Output:

METHOD	TOTAL_AMOUNT
Net Banking	61000
Card	77000
Cash	4000
UPI	44000

Q12. Display the names of customers who gave a rating of 5.

```
SELECT C.Name, R.Rating FROM CUSTOMER C  
  
JOIN REVIEW R  
ON C.Customer_ID = R.Customer_ID  
WHERE R.Rating = 5;
```

Output:

NAME	RATING
Ananya	5
Sneha	5
Priya	5
Ravi	5
Pooja	5
Meena	5
Harini	5

Q13. Display products whose stock is greater than the average stock of all products.

```
SELECT Name, Stock FROM PRODUCT
WHERE Stock > (SELECT AVG(Stock) FROM PRODUCT);
```

Output:

NAME	STOCK
T Shirt	30
Running Shoes	25
DBMS Book	40
Face Cream	35
Rice Bag	50
Backpack	28
Wireless Mouse	32

Q14. Display the names of products that received the highest rating.

```
SELECT P.Name, R.Rating FROM PRODUCT P
JOIN REVIEW R
ON P.Product_ID = R.Product_ID
WHERE R.Rating = (SELECT MAX(Rating) FROM REVIEW);
```

Output:

NAME	RATING
Smart TV	5
Dell Laptop	5
Running Shoes	5
Cricket Bat	5
Wrist Watch	5
Toy Car	5
Wireless Mouse	5

Q15. Display the product having the lowest available stock.

```
SELECT Name, Stock
FROM PRODUCT
WHERE Stock = (SELECT MIN(Stock) FROM PRODUCT);
```

Output:

NAME	STOCK
Dell Laptop	10

11. CONCLUSION

The Online Shopping System provides a structured relational database solution for managing online shopping-related information. The design organizes customers, products, orders, payments, deliveries, reviews, and cart details into related relations and applies primary keys, foreign keys, uniqueness rules, NOT NULL conditions, and CHECK constraints to support data integrity. The normalization process up to 3NF reduces redundancy and update anomalies. The SQL implementation and query set demonstrate how the database can support insertion, modification, retrieval, and reporting of online shopping-related information. The project therefore provides a suitable DBMS implementation that can be extended into a complete online shopping management system.



12. FUTURE ENHANCEMENTS

- Role-based access for administrators and authorized users.
- Online product browsing, selection, and ordering.
- Online product cart management and order tracking.
- Order scheduling and automated delivery notifications.
- Online payment integration and payment tracking.
- Automated SMS/email notifications for order status.
- Product, order, and sales status tracking and report generation.
- Backup, recovery, audit logging, and stronger database security.

